数字逻辑基础与Verilog设计
---组合篇

第3章 算术运算电路

本章内容

- 无符号算术运算
- 有符号算术运算
  - 加减溢出检测；
  - 无符号数加减器、用作比较器、有符号数乘法；

10. 有符号数的算术运算

- 有符号数：
  - n位有符号数，最左边的一位表示符号，0为整数，1为负数，剩余的n-1位表示数值大小，$B = b_{n-1}b_{n-2}\cdots b_1b_0$
  - 三种表示方法：原码，反码，补码。

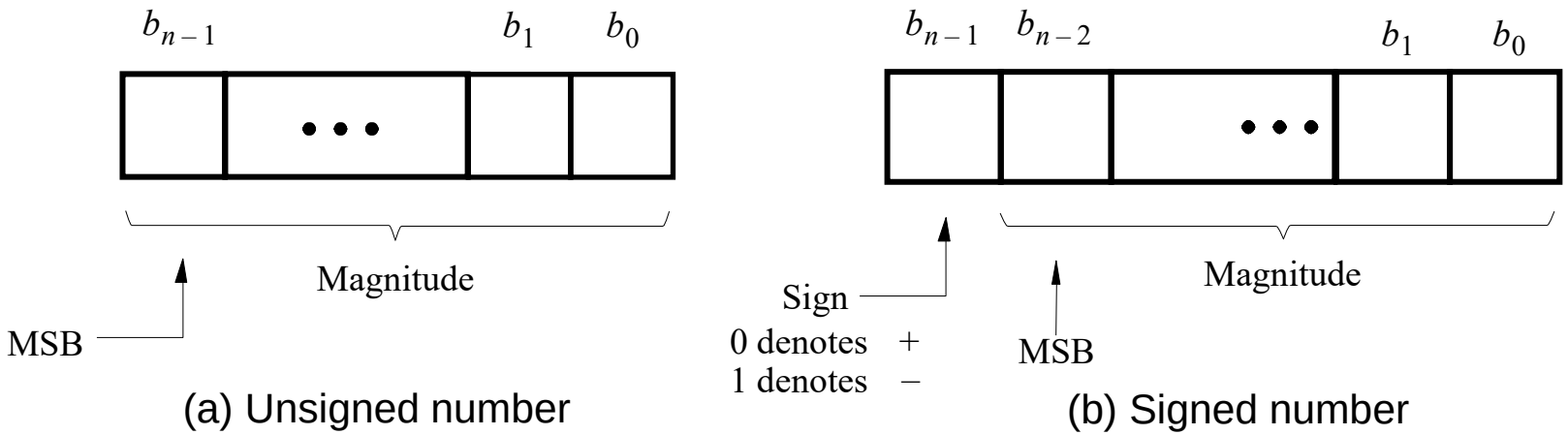


Figure 3.7. Formats for representation of integers.

10. 有符号数的算术运算

- 加减溢出的判定：（*A*、*B*为补码形式）
  - $A = a_{n-1}a_{n-2}\cdots a_1a_0$, $B = b_{n-1}b_{n-2}\cdots b_1b_0$, 和$S = s_{n-1}s_{n-2}\cdots s_1s_0$;
  - n位有符号数，范围是：$-2^{n-1}\sim 2^{n-1}-1$。运算结果超出机器数所能表示的范围，就会溢出。
    - 两个异号数相加或两个同号数相减，不会溢出。
    - 两个同号数相加或两个异号数相减，有可能溢出

进位: c_4 c_3
c_3:最高位进位
c_4:符号位进位

① 9+3=12	② 11+7=18	③ -6-7=-13	④ -11-7=-18
0 1001	0 1011	1 1010	1 0101
+ 0 0011	+ 0 0111	+ 1 1001	+ 1 1001
0 0	0 1	1 1	1 0
0 0 1100	0 1 0010	1 1 0011	1 0 1110
$c_3=c_4$, 无溢出	$c_3\neq c_4$, 溢出	$c_3=c_4$, 无溢出	$c_3\neq c_4$, 溢出

10. 有符号数的算术运算

- 加减溢出：
  - 溢出条件1：**Overflow** = $c_n \oplus c_{n-1}$

    解释：任意符号的两数相加，数值位最高位的进位c_{n-1}和符号位的进位c_n值不同，则溢出。
  - 溢出条件2：**Overflow** $= a'_{n-1} b'_{n-1} s_{n-1} + a_{n-1} b_{n-1} s'_{n-1}$

    解释：符号相同的两数相加，结果的符号与加数不同，则溢出。

10. 有符号数的算术运算

- 溢出条件3：加数采用双符号位（正数00，负数11），结果的两符号位不同时，则溢出。
- 例：

① 9+5=14

$$\begin{array}{r} 00\ 1\ 0\ 0\ 1 \\ +\ 00\ 0\ 1\ 0\ 1 \\ \hline 00\ 1\ 1\ 1\ 0 \end{array}$$

无溢出

② 12+7=19

$$\begin{array}{r} 00\ 1\ 1\ 0\ 0 \\ +\ 00\ 0\ 1\ 1\ 1 \\ \hline 01\ 0\ 0\ 1\ 1 \end{array}$$

溢出

10. 有符号数的算术运算

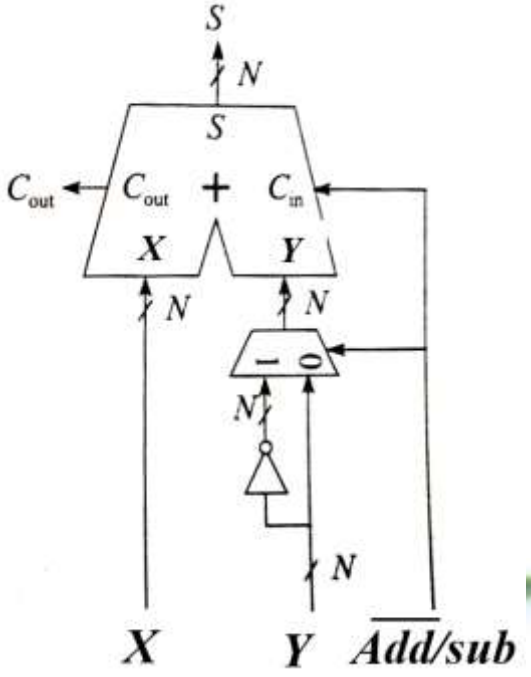
- 应用实例5：n位有/无符号数的加/减器: $x \pm y$
- 有符号整数与无符号整数加减运算电路是完全一样的。若是有符号数，则 X 与 Y 表示 x 和 y 的补码；

  减法：无论补码减法还是无符号数减法：被减数加上减数相反数的补码来实现。

$$[x-y]_{补}=[x]_{补}+[-y]_{补}=X+\overline{Y}+1$$

  加法：$x+y = X+Y$。

- 引入$\overline{Add/sub}$作为加/减法控制信号，增加二选一MUX。



10. 有符号数的算术运算

- 也可以采用“异或”门代替二选一数据选择器。

□思考：无符号整数加减运算有哪些应用？（地址或指针运算）

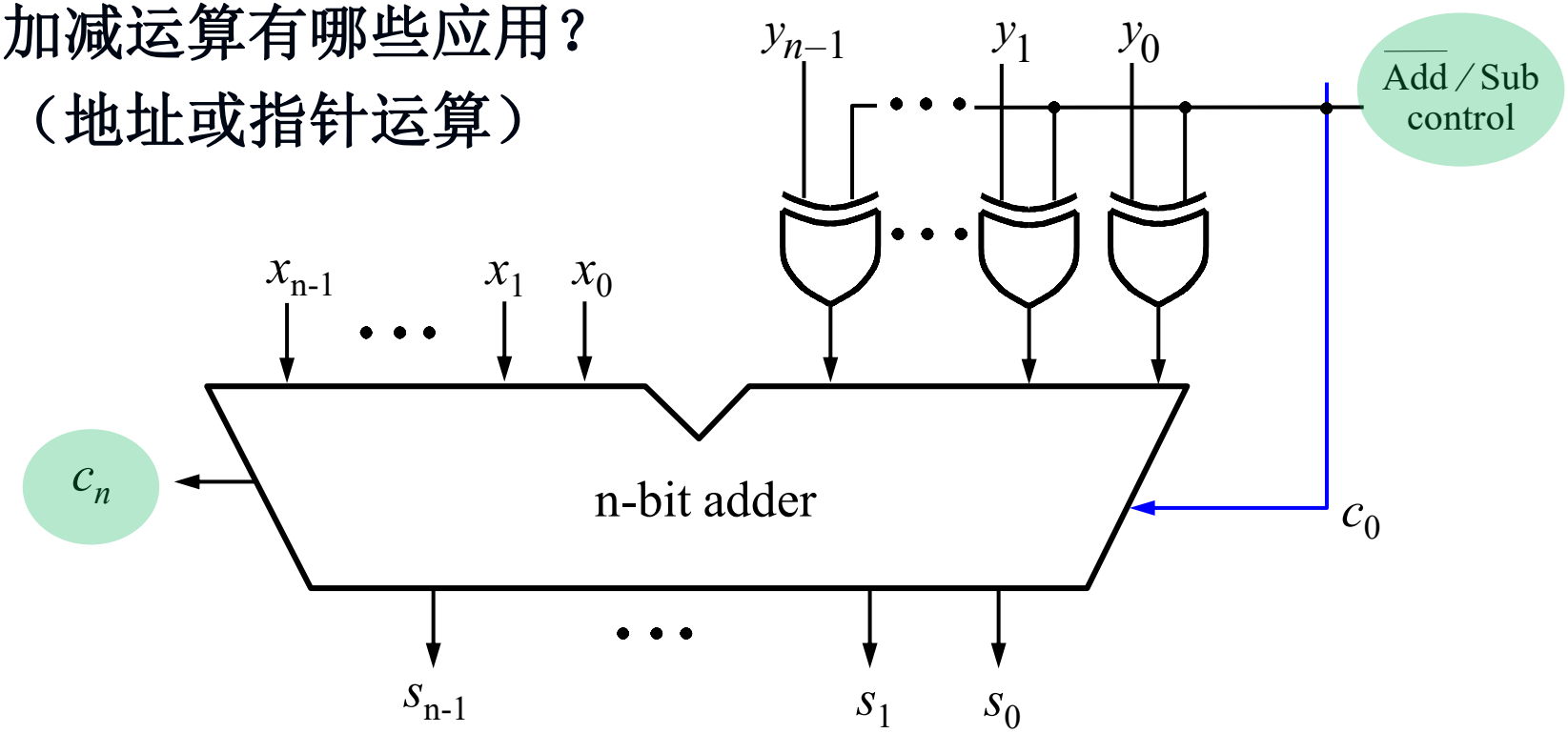
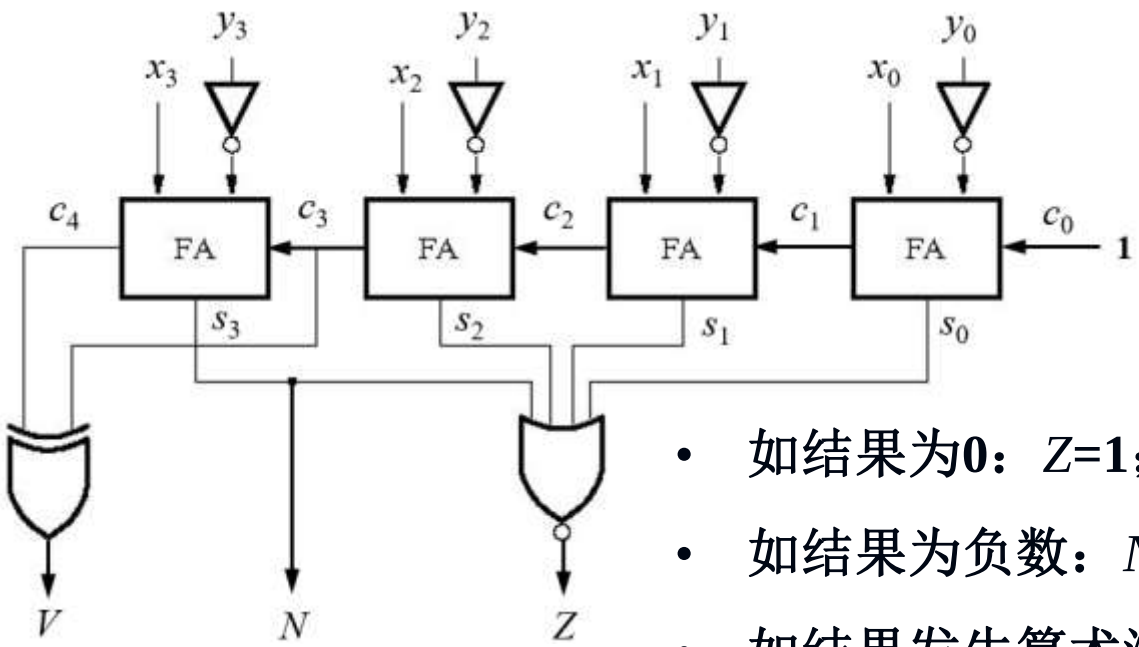


Figure 3.12. Adder/subtractor unit.

10. 有符号数的算术运算

- 应用实例6：利用加法器实现有符号数的数值比较器
- 有符号数以补码给出：$X= x_{n-1}x_{n-2}\cdots x_1x_0$, $Y= y_{n-1}y_{n-2}\cdots y_1 y_0$
- 下图实现：X-Y。3个输出信号：Z, N, V的意义如下：



- 如结果为0：Z=1；否则Z=0；
- 如结果为负数：N=1；否则N=0；
- 如结果发生算术溢出：V=1；否则V=0；

10. 有符号数的算术运算

- 应用实例6：利用加法器实现有符号数的数值比较器
- 如何利用Z、N、V 判断 X 和 Y 的大小关系？
- (1) $X < Y$：若 X 和 Y 符号相同，不溢出V=0，差为负N=1；

  若 X 为负数，Y 为正数: 若不溢出 V=0，差为负数 N=1；若溢出 V=1，差为正数 N=0。

$$XltY = N \oplus V$$

- (2) $X = Y$：结果为0，$Z = 1$；
- (3) $X > Y$：$XgtY = \overline{N \oplus V + Z}$

10. 有符号数的算术运算

- 应用实例7：n位有符号数的乘法: *X *Y*

- 采取原码运算:
  - 符号位（异或）与数值位（无符号数运算）分别处理；
  - 若数据为补码形式，需要两次补码与原码间的转换。
- 采用补码运算：
  - 从补码开始，直接得到补码的积；
  - 比较好的方法是Booth比较法。

10. 有符号数的算术运算

- 应用实例7：n位有符号数的乘法: *X *Y*
- 补码一位乘法的Booth算法：

  举例：乘数*Y* = 00101110，46=64−32+16−2

  - *Y*= 01000000 – 00100000 + 00010000 – 00000010
  - *X*Y=X*01000000-X*00100000+X*00010000-X*00000010*

- Booth算法将乘数看作从最低位开始的一串二进制数。从最低位算起，若这串数字为“0”，不做任何操作；若遇到第一个“1”时执行一次减法，即减被乘数与该位权值的乘积，而对于其后的“1”不执行任何操作；若遇到第一个“0”时执行一次加法，即加被乘数与该位权值的乘积，而对其后的“0”则不做任何操作。如此一直进行到最高位。
- 例：5*11=00000101*00001011，结果：$0-5*2^0+5*2^2-5*2^3+5*2^4=55$

10. 有符号数的算术运算

- 应用实例7：n位有符号数的乘法: $X*Y$
- 补码一位乘法的Booth算法：

举例：乘数Y= 00101110，46=64−32+16−2

- Y= 01000000 – 00100000 + 00010000 – 00000010
- $X*Y=X*01000000-X*00100000+X*00010000-X*00000010$
- $X*Y=X*0(+1)000000+X*00(-1)00000+X*000(+1)0000+X*000000(-1)0$
- $X*Y=X*$ 0 (+1) (-1) (+1) 0 0 (-1) 0

对照乘数：Y= 0 0 1 0 1 1 1 0′ 0

解释：对乘数Y低位增加一位辅助位y_{-1}，初值为0，从低到高，依次两位作差，对Y重新编码。

10. 有符号数的算术运算

- 应用实例7：n位有符号数的乘法: $X * Y$

- 再如:乘数（负数）Y =10010010，-110= -128+32-16+4-2，对Y重新编码：(-1)0 (+1) (-1) 0 (+1) (-1) 0

补码一位乘法的Booth算法描述：

被乘数X与乘数Y均以补码形式参加运算(符号位参与运算)，运算结果是积的补码：$[X]_{补}= x_{n-1}x_{n-2}\cdots x_1x_0$, $[Y]_{补}= y_{n-1}y_{n-2}\cdots y_1y_0$

算法步骤:

- 乘数补码末位y_0之后增加一位附加位y_{-1}，初始时$y_{-1}= 0$；
- 部分积P为2n+1位：P[2n:0]，初始化部分积为0；
- 将乘数Y填入 P[n:1]中;

10. 有符号数的算术运算

- 应用实例7：n位有符号数的乘法: $X*Y$

补码一位乘法的Booth算法描述（接）：

- 从 P向量的最低位依次向左扫描，并判断扫描的两位二进制数，根据Y_iY_{i+1}的值，按照下列规则进行累加与右移操作，右移时遵循补码的移位规则（最高位以符号位填充）；
  - 如果 $y_0\,y_{-1}$ =00或11，则无操作；
  - 如果 $y_0\,y_{-1}$= 01，则部分积加$[X]_{补}$；
  - 如果 $y_0\,y_{-1}$= 10，则部分积加$[-X]_{补}$；
- 判断p[2n]，若为 0，右移一位补 0；若为1，右移一位补 1;
- 重复上述步骤，循环n次。
- P[2n:1]为最终的乘积。

10. 有符号数的算术运算

例：被乘数A 为7 (0111)，乘数B为2 (0010) ， n=4，P的容量是n*2+1=9.

开始先求出“-A”的补码A，初始化P 空间 P空间的[4..1] 填入乘数	A = 0111，A= 1001，P = 0000 0000 0 P = 0000 0010 0
判断P[1:0]，是2'b00 亦即“无操作” 判断P[8], 若是0右移一位，补0；反之补1。	P = 0000 0010 0 P = 0000 0001 0
判断P[1:0]，是2'b10 亦即“-被乘数”。 P空间的[8..5] 和 被乘数 A相加。 判断P[8], 若是0右移一位，补0；反之补1	P = 0000 0001 0 + 1001 P = 1001 0001 0 P = 1100 1000 1
P判断P[1:0]，是2'b01 亦即“+被乘数”。 空间的[8..5] 和 被乘数 A 相加。 判断P[8], 若是0右移一位，补0；反之补1	p = 1100 1000 1 + 0111 P = 0011 1000 1 无视最高位溢出 P = 0001 1100 0
判断P[1:0]，是2'b00 亦即“无操作” 判断P[8], 若是0右移一位，补0；反之补1	P = 0001 1100 0 P = 0000 1110 0
最终P空间的[8..1] 就是最终答案。	P = 0000 1110

例：被乘数X为13 (01101)，乘数Y为-10 (10110)，n=5，P[10:0]

开始先求出(-X)，初始化P 空间 P空间的[5..1] 填入乘数Y	X = 01101，X= 10011，P = 11'd0 P = 00000 10110 0
判断P[1:0]，是2'b00, 亦即“无操作” 判断P[10], 若是0右移一位，补0；反之补1。	P = 00000 01011 0
判断P[1:0]，是2'b10 亦即“-被乘数”。 P空间的[10..6] 和 被乘数 X相加。 判断P[10], 若是0右移一位，补0；反之补1	P = 00000 01011 0 + 10011 P = 10011 01011 0 P = 11001 10101 1
判断P[1:0]，是2'b11, 亦即“无操作” 判断P[10], 若是0右移一位，补0；反之补1	P = 11100 11010 1
判断P[1:0]，是2'b01 亦即“+被乘数”。 空间的[10..6] 和 被乘数 X 相加。 判断P[10], 若是0右移一位，补0；反之补1	p = 11100 11010 1 + 01101 P = 01001 11010 1 无视最高位溢出 P = 00100 11101 0
判断P[1:0]，是2'b10 亦即“-被乘数” 空间的[10..6] 和 被乘数 X 相加。 判断P[10], 若是0右移一位，补0；反之补1	P = 00100 11101 0 + 10011 P = 10111 11101 0 P = 11011 11110 1
最终P空间的[10..1] 就是最终答案。	P = 11011 11110

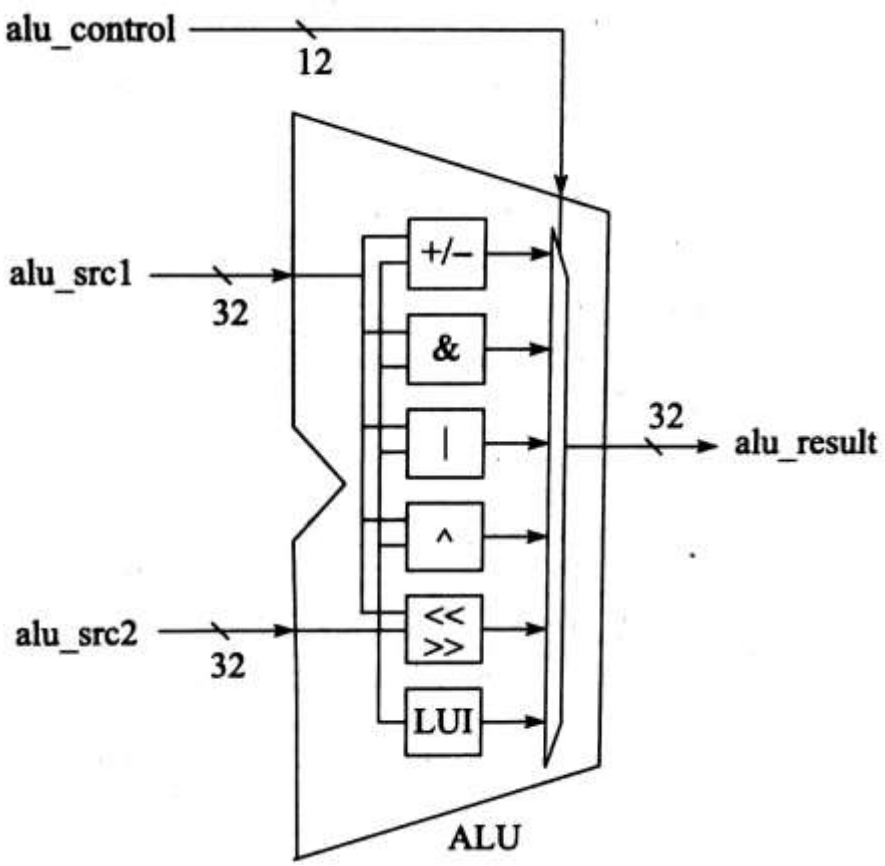
简单的ALU实验设计案例

端口说明：

- alu_control: 控制信号，独热码；
- alu_src1和alu_src2: 运算操作数;
- alu_result: 32位运算结果;

功能说明：

- +/-:加减运算；输出
- &，|，^: 按位与、或、异或运算；
- >>, <<: 逻辑移位；
- 算术右移；
- 有（无）符号数的比较运算；
- LUI：置高位立即数；



简易ALU电路结构

简单的ALU实验设计

Verilog代码：

```
module simple_alu(
        input  [11:0]  alu_control,
        input  [31:0]  alu_src1,
        input  [31:0]  alu_src2,
        output  [31:0]  alu_result
);
wire op_add;  // 加法操作
wire op_sub;  // 减法操作
wire op_slt;   // 有符号数比较，小于置位
wire op_sltu;  // 无符号数比较，小于置位
```

```
wire op_and; // 按位与
wire op_or;   //按位或
wire op_xor; //按位异或
wire op_nor; //按位或非
wire op_sll;   //逻辑左移
wire op_srl;  // 逻辑右移
wire op_sra; //算术右移
wire op_lui;  // 高位加载
```

简单的ALU实验设计

```
assign op_add  = alu_control[ 0];
assign op_sub  = alu_control[ 1];
assign op_slt  = alu_control[ 2];
assign op_sltu = alu_control[ 3];
assign op_and  = alu_control[ 4];
assign op_or   = alu_control[ 5];
assign op_xor  = alu_control[ 6];
assign op_nor  = alu_control[ 7];
assign op_sll  = alu_control[ 8];
assign op_srl  = alu_control[ 9];
assign op_sra  = alu_control[10];
assign op_lui  = alu_control[11];
```

```
wire [31:0] add_sub_result;
wire [31:0] slt_result;
wire [31:0] sltu_result;
wire [31:0] and_result;
wire [31:0] or_result;
wire [31:0] xor_result;
wire [31:0] nor_result;
wire [31:0] sll_result;
wire [31:0] srl_result;
wire [31:0] sra_result;
wire [31:0] lui_result;
```

简单的ALU实验设计

```
assign and_result = alu_src1 & alu_src2;
assign or_result  =  alu_src1 | alu_src2;
assign nor_result  =  ~ or_result ;
assign xor_result  = alu_src1 ^ alu_src2;
assign lui_ result  = {alu_src2[15:0],16'b0};

wire [31:0] adder_a;
wire [31:0] adder_b;
wire        adder_cin;
wire [31:0] adder_result;
wire        adder_cout;
```

简单的ALU实验设计

```
assign adder_a    =  alu_src1;
assign adder_b    =  (op_sub | op_slt | op_sltu) ? ~alu_src2 :  alu_src2;
assign adder_cin =  (op_sub | op_slt | op_sltu) ? 1'b1 :  1'b0;
assign {adder_cout, adder_result} = adder_a + adder_b + adder_cin ;
assign add_sub_result = adder_result; //加减运算结果
assign slt_result[31:1] = 31'b0;
assign slt_result[0] = (alu_src1[31] & ~alu_src2[31])
                        | (~(alu_src1[31] ^ alu_src2[31]) & adder_result[31]);
// 有符号数比较结果：src1<0且src2>0 或 同符号数且作差结果为负数
assign sltu_result[31:1] = 31'b0;
assign sltu_result[0] = ~adder_cout; // 无符号数比较结果
```

简单的ALU实验设计

```
assign sll_result =  alu_src2 << alu_src1[4:0];  //逻辑左移alu_src1[4:0]位
assign srl_result =  alu_src2 >> alu_src1[4:0]; //逻辑右移alu_src1[4:0]位
assign sra_result = ($signed(alu_src2)) >>> alu_src1[4:0];
// 算术移位结果：利用“$signed” 内置函数和运算符号“>>>”完成
assign alu_result = ({32{op_add | op_sub}} & add_sub_result)
                  | ({32{op_slt}} & slt_result) | ({32{op_sltu}} & sltu_result)
                  | ({32{op_and }} & and_result) | ({32{op_nor}} & nor_result)
                  | ({32{op_or}} & or_result) | ({32{op_xor}} & xor_result)
                  | ({32{op_sll}} & sll_result) | ({32{op_srl}} & srl_result)
                  | ({32{op_sra}} & sra_result) | ({32{op_lui}} & lui_result);
endmodule
```

本章作业

- 验证3.8，3.9
- 3.14，3.16，3.17,
- 3.18，3.19，3.23,3.25(选做)